

NOC Implementation Plan

Dynamic MTR Traceroute Monitoring System

Dokumen ini menguraikan arsitektur dan langkah implementasi teknis untuk membangun sistem monitoring jaringan berbasis **MTR (My Traceroute)**. Sistem ini dirancang secara dinamis, memungkinkan penambahan atau penghapusan IP tujuan tanpa menghentikan layanan. Menjalankan *backend* Python ini pada *server* yang tangguh dengan sistem operasi seperti Ubuntu 24.04 (misalnya pada mesin PowerEdge R620) akan memastikan eksekusi MTR paralel berjalan dengan sangat stabil dan optimal.

1. Arsitektur Sistem

- **Data Collector (Backend):** Python mengeksekusi `mtr -j` secara paralel menggunakan *multi-threading* atau *asyncio*. IP target dibaca secara dinamis (misalnya dari tabel database lokal SQLite atau endpoint API).
- **Time-Series Database:** InfluxDB bertugas menyimpan metrik performa (Loss, Avg, Worst, StDev) per *hop* dan per destinasi.
- **Data Provider (API):** Python (FastAPI/Flask) membaca data dari InfluxDB dan menyajikannya dalam format JSON untuk *frontend*.
- **Dashboard (Frontend):** Antarmuka web kustom menggunakan Tailwind CSS, Vis.js (Topology), dan Chart.js/D3.js (Grafik) untuk memvisualisasikan data dari API.

2. Manajemen Target Dinamis

Agar IP destinasi bersifat dinamis (bisa ditambah/dihapus kapan saja tanpa *restart script*), kita akan menggunakan pendekatan **Dynamic Watcher**. Python akan menyimpan daftar IP dalam *database* lokal (SQLite) atau InfluxDB. Setiap kali siklus *scanning* dimulai (misal tiap 10 detik), Python melakukan query IP target aktif terlebih dahulu.

3. Desain Skema Database (InfluxDB)

Format data JSON yang dihasilkan oleh `mtr -j` menyediakan parameter lengkap (Loss%, Snt, Last, Avg, Best, Wrst, StDev). Struktur *bucket* dan *measurement* pada InfluxDB akan didesain sebagai berikut:

Measurement	Tags (Indeks Pencarian)	Fields (Nilai Metrik)
mtr_hop_stats	target_ip, hop_number, hop_ip	loss_pct, avg_rtt, worst_rtt, stdev_rtt
mtr_end_to_end	target_ip	total_hops, end_loss_pct, end_avg_rtt

4. Implementasi Backend (Python)

Backend akan menggunakan modul `subprocess` untuk memanggil *binary* MTR bawaan Ubuntu. Proses eksekusi akan dibuat paralel agar tidak ada waktu tunggu (*blocking*) jika ada IP yang RTO.

```
# Contoh snippet eksekusi MTR & Parsing ke JSON
import subprocess
import json

def run_mtr(ip_address):
    # -r: Report, -j: JSON output, -c 5: Kirim 5 ping per hop
    result = subprocess.run(
        ['mtr', '-r', '-j', '-c', '5', ip_address],
        capture_output=True, text=True
    )
    if result.returncode == 0:
        mtr_data = json.loads(result.stdout)
        # Parse data dan Push ke InfluxDB
        return mtr_data['report']
```

5. Visualisasi Dashboard (Frontend Tailwind)

Dashboard akan dibangun dengan Tailwind CSS untuk tata letak (*layouting*) yang responsif. Sesuai spesifikasi, halaman akan dibagi menjadi beberapa segmen visualisasi utama:

A. Topologi Jaringan Terpadu (Vis.js)

Posisi: Baris pertama (Row 1), lebar penuh (Full-width 100%).

Logika: Vis.js akan membaca seluruh jalur dari *source* Ubuntu ke **semua** IP Destinasi. Node yang sama (misal *Gateway* ISP yang dilewati beberapa jalur sekaligus) akan digabung secara visual, menghasilkan diagram pohon (*tree/mesh*) yang interaktif (dapat di-*zoom* dan di-*drag*).

B. Panel Metrik & Grafik Performa

Di bawah topologi, *layout* dibagi menjadi kolom-kolom grafik (Grid system Tailwind) menggunakan Chart.js atau D3.js:

- **Average Total Hops (Vertical Bar Chart):**

Menampilkan batang vertikal untuk setiap IP destinasi. Tinggi batang menunjukkan rata-rata jumlah *hop* yang dibutuhkan untuk mencapai IP tersebut.

- **End-to-End Packet Loss (Line Chart):**

Menampilkan tren persentase *packet loss* pada titik akhir (destinasi) dari waktu ke waktu. Jika garis naik di atas 0%, terindikasi koneksi terputus/tidak stabil.

- **Worst Latency (Line Chart):**

Mengambil *field* `wrst` dari hasil MTR. Grafik garis ini akan menampilkan lonjakan latensi tertinggi (*latency spikes*) untuk setiap IP destinasi. Cocok untuk menginvestigasi jitter parah pada jaringan.

- **Standar Deviasi RTT (Line/Area Chart):**

Mengambil *field* `StDev` dari MTR. Grafik ini menunjukkan fluktuasi/kestabilan jaringan. Nilai deviasi yang tinggi (grafik naik) menunjukkan bahwa latensi sangat tidak stabil (*ping lompat-lompat*).

6. Rencana Eksekusi (Langkah Implementasi)

1. **Fase 1: Persiapan OS & Database.** *Update repository*, `apt install mtr`, dan pasang InfluxDB (via Docker direkomendasikan). Buat *bucket* dan *token* akses.
2. **Fase 2: Backend Development.** Buat *script* Python `mtr_collector.py`, hubungkan dengan InfluxDB client, uji coba penambahan IP dinamis via JSON file/SQLite.
3. **Fase 3: API Gateway.** Buat *mini-server* menggunakan FastAPI untuk menjembatani *query* dari Frontend ke InfluxDB.
4. **Fase 4: Frontend Development.** Rancang struktur HTML/Tailwind. Integrasikan Vis.js untuk parsing *hop list* menjadi *nodes & edges*. Implementasikan grafik metrik deret waktu.
5. **Fase 5: Systemd Deployment.** Konfigurasi Python *backend* menjadi *background service* di Linux (seperti `noc-mon.service`) agar otomatis berjalan dan anti-mati.